SOURCE OF TRUTH

Benchmarking Post-Quantum Cryptography in Cloud Object Stores under Scaled Client Concurrency

ML-KEM-768 + HKDF-SHA256 + AES-256-GCM envelope encryption over an S3-compatible MinIO store — requirements, decisions, protocol, experiment design and plan

Field	Value
Project	Team 12 research paper assignment (4 students)
Document	Project Source of Truth (SoT), version 1.4 (change log in §1.4)
Date	24 September 2026
Supersedes	Implementation Specification v1.0 (pqc-cloud-implementation.pdf) where they conflict; corrects the draft paper (pqc-cloud-object-store-benchmark.pdf)
Subordinate to	Professor's assignment brief (Team12_Paper_Assignment) — the grading contract
Status	Baseline for implementation. Changes only through the change-control rule in §1.3

This document consolidates three inputs — the assignment brief, the team's implementation specification and the team's draft paper — with independent research verified on 24 September 2026. Where the inputs disagree, this document records a single decision and the reason for it. Code, experiments and paper text must conform to it.

Contents

1. How to use this document . . . 4
1.1 Purpose . . . 4
1.2 Order of precedence . . . 4
1.3 Change control . . . 4
1.4 Change log . . . 4
2. Project summary . . . 5
2.1 Objective . . . 5
2.2 What the system does, in one paragraph . . . 5
2.3 Contribution the paper will claim (subject to results) . . . 5
3. Requirements traceability . . . 6
4. Verified research findings (as of 24 Sep 2026) . . . 6
4.1 ML-KEM parameters (FIPS 203, published 13 Aug 2024) . . . 6
4.2 MinIO community edition is archived . . . 7
4.3 Library status and versions . . . 7
4.4 Reference micro-benchmarks (indicative only) . . . 7
4.5 Prior art that bounds our novelty claims . . . 8
5. Binding design decisions . . . 8
6. Reconciliation of the three input documents . . . 9
6.1 Conflicts between documents and how they are resolved . . . 9
6.2 Errata for the draft paper (must be fixed before reuse) . . . 10
6.3 Amendments to the implementation specification . . . 10
7. Cryptographic protocol (normative) . . . 11
7.1 Roles and keys . . . 11
7.2 Object identity and authenticated context . . . 11
7.3 PUT — per object (mode P1) . . . 11
7.4 GET — per object (mode P1) . . . 11
7.5 Stored bytes per object, by mode . . . 12
7.6 Mode definitions . . . 12
7.7 Nonce, key and secret-handling rules . . . 12
8. Metrics, timers and records . . . 13
8.1 The four assignment metrics — exact definitions . . . 13
8.2 Timer decomposition . . . 13
8.3 Record schemas . . . 13
8.4 Derived metrics . . . 14
9. Experiment design . . . 14
9.1 Pre-registered research questions and hypotheses . . . 14
9.2 Experiment matrix (replaces the spec's final matrix) . . . 14

9.3 Run protocol . . . 15
9.4 Time and resource budget . . . 15
10. Infrastructure and measurement environment . . . 15
11. Correctness and security tests (gate for all measurements) . . . 16
12. Analysis plan . . . 17
12.1 Statistics . . . 17
12.2 Queueing model (RQ3) . . . 17
12.3 Planned figures and tables . . . 17
13. Repository and tooling . . . 18
14. Team roles and schedule . . . 18
14.1 Ownership (4 students) . . . 18
14.2 Schedule (relative weeks — align W10 with the submission deadline) . . . 18
15. Definition of done . . . 19
16. Risks, threats to validity and open questions . . . 19
16.1 Risk register . . . 19
16.2 Threats to validity (to be written into the paper) . . . 19
16.3 Questions to confirm with the professor (W1) . . . 20
17. References . . . 20

1. How to use this document

1.1 Purpose

One place that answers: *what are we building, why, exactly how, how will we measure it, who does what, and what counts as done.* Every design choice in code, every experimental parameter and every claim in the paper must be traceable to a section here. If something is not here, it is not yet decided.

1.2 Order of precedence

Rank	Source	Role
1	Assignment brief (professor)	Non-negotiable requirements: MinIO, liboqs, Kyber-768 (ML-KEM-768), AES-256-GCM, 1–500 workers, 1 MB–500 MB, RSA-4096 and AES-only baselines, four metrics.
2	This Source of Truth	Binding decisions, protocol, experiment design, schedule.
3	Implementation Specification v1.0	Detailed engineering guidance; valid unless amended in §6.3.
4	Draft paper	Background and framing only; must be corrected per §6.2 before reuse.

1.3 Change control

- Any change to a decision (D-numbers), the manifest schema, timer definitions or the experiment matrix requires a pull request that edits this document's source and is approved by at least two team members.
- Once final measurements start (Phase 8), the protocol, dependencies and matrix are **frozen**. A change after that point invalidates previously collected final data.
- Numbers in the paper come only from versioned analysis scripts run on retained raw files — never typed in by hand and never taken from pilot runs.

1.4 Change log

Version	Date	Change	Status
1.0	24 Sep 2026	Initial baseline.	Approved (Revanth)
1.1	24 Sep 2026	Aligned with the Phase 2 implementation: (1) object versions are UUIDv7 instead of UUIDv4, so the latest committed version can be found by listing (§7.2); (2) the manifest nests the authenticated context as a "context" object next to ke_ct, salt and nonce_prefix (§7.3); (3) liboqs is built with OQS_MINIMAL_BUILD limited to ML-KEM-512/768/1024 and capped build parallelism (D12); (4) the one-shot v1 format was never built: milestone M1 already uses the chunked format (D7, A4); (5) implementations must warm up each process before timed operations; the first liboqs call costs ~70 ms of library loading (§8.2).	Pending team review (§1.3)
1.2	24 Sep 2026	Phase 4 measurement: per-transfer client memory (§9.4) replaced the estimate with measured values (~50 MiB per large in-flight transfer, independent of object size).	Pending team review (§1.3)
1.3	24 Sep 2026	Phase 5 implementation: (1) timer-validation rule is per run: no request may show negative unaccounted time (double counting), and per run the p99 of unaccounted/total must be ≤ 2% and the aggregate ≤ 1%; single requests delayed by the OS are counted as outliers, not failures (§8.2, §12.1); (2) worker processes disable the garbage collector during the timed window; (3) object identity creation is timed as SER and the AES key schedule as KDF, so no per-request work is unattributed; (4) the benchmark HTTP client never retries: failures are recorded, not hidden in latency; (5) server CPU/RSS/network are scraped from MinIO's own /minio/v2/metrics/node endpoint, no agent needed (§10); (6) readers must name the expected mode, so a planted B0 object cannot downgrade P1 (§7.6).	Pending team review (§1.3)

Version	Date	Change	Status
1.4	24 Sep 2026	Phase 7 (N1): (1) N1 context adds authenticated session_id, session_seq and session_bound; HKDF domain 'pqc-s3-session-v1'; the bound is fixed per session at creation, so B_max and T_max are hard caps (§16); (2) queue-aware utilization = in-flight requests / worker threads of the process, EWMA-smoothed on every grant; (3) receivers cache decapsulated roots under their own B_max and T_max and reject objects beyond their B_max; (4) prior-art search found S3 Bucket Keys and the AWS Encryption SDK cache: bounded reuse itself is not claimed as new; the queue-aware policy and the Sweep E trade-off evaluation are (docs/n1-prior-art.md, §4.5); (5) config/n1.yaml implements Sweep E.	Pending team review (§1.3)

2. Project summary

2.1 Objective

Quantify the performance cost of protecting objects in an S3-compatible store with NIST's standardized post-quantum key-encapsulation mechanism **ML-KEM-768** (FIPS 203; formerly CRYSTALS-Kyber-768), compared with classical key establishment (**RSA-4096**, X25519) and with symmetric-only protection (**AES-256-GCM**), as client concurrency scales from 1 to 500 and object sizes scale from 1 KiB to 500 MiB.

2.2 What the system does, in one paragraph

A client encrypts every object *before* it reaches the object store. For each object it runs ML-KEM-768 encapsulation against a recipient's public key, obtaining a 1,088-byte KEM ciphertext and a 32-byte shared secret. HKDF-SHA256 turns that secret and the object's identity into a unique AES-256 key. The object is encrypted in fixed-size authenticated chunks with AES-256-GCM, so memory stays bounded even for 500 MiB objects. The ciphertext stream and a small manifest (KEM ciphertext, salt, nonce prefix, authenticated context) are written to MinIO. Download reverses the process; decryption releases no plaintext unless every authentication tag verifies. Each stage is timed separately, and the same data path is re-run with each baseline, so differences can be attributed to key establishment alone.

2.3 Contribution the paper will claim (subject to results)

- **C1 — Object-level PQC benchmark.** An instrumented, reproducible benchmark of per-object ML-KEM-768 envelope encryption on an S3-compatible store with matched classical baselines, stage-level latency decomposition, client and server resource profiles, and scaling to 500 concurrent clients.
- **C2 — Predictive model.** A service-time decomposition and queueing model that predicts where the crypto client saturates, validated against open-loop measurements with reported prediction error.
- **C3 — Bounded KEM amortization (stretch).** A session mode that reuses one KEM-established root for at most B_{max} objects / T_{max} seconds with per-object HKDF separation, evaluated as a performance-versus-compromise-scope trade-off, plus an optional queue-aware bound. Positioned against existing data-key caching (see §4.5).

3. Requirements traceability

Every item in the assignment brief, mapped to where this project satisfies it. "Modified" means the requirement is met with a documented design adjustment that the team should confirm with the professor (see §16).

ID	Assignment requirement	How satisfied	Status
R1	Deploy a local MinIO object storage cluster on Linux instances	MinIO built from source at final release tag, run on a dedicated Linux VM (D1, §10). Single node for the main matrix; optional 4-node distributed run as extension.	Modified
R2	Client using liboqs wrapping PutObject/GetObject with Kyber-768 KEM + AES-256-GCM	pqcstore client: liboqs-python 0.16.0.1 over liboqs 0.16.0; ML-KEM-768; HKDF-SHA256; chunked AES-256-GCM (§7).	Covered
R3	Stress tests scaling concurrency 1 → 500 workers	Concurrency levels 1, 8, 32, 64, 128, 256, 500 via multi-process runner (§9, §10).	Covered
R4	File sizes 1 MB → 500 MB	1 MiB, 16 MiB, 128 MiB, 500 MiB (+1 KiB, 64 KiB to expose fixed KEM cost). Full 500×500 cross-product replaced by three sweeps (D8).	Modified
R5	Profile host resource usage and latency breakdowns	Per-request stage timers + 1 Hz client and server samplers (§8.3).	Covered
R6	Baseline: classical RSA-4096 key exchange	Mode R1: RSA-OAEP-4096 key transport of a 32-byte secret, then identical HKDF+AES path (D4).	Added (missing in spec)
R7	Baseline: AES-256-GCM without PQC	Mode B1 (§7.6).	Covered
R8	Metric: cryptographic handshake overhead (ms)	T_{KE} + T_{KDF} per object, per operation (§8.1).	Covered
R9	Metric: data throughput (MB/s)	Successful plaintext bytes / measured interval (§8.1).	Covered
R10	Metric: client/server CPU utilization (%)	Client process-tree and MinIO process sampling; CPU-ms per request (§8.1).	Added (server side)
R11	Metric: memory footprint growth (MB)	Peak − idle RSS, client and server; metadata bytes per object (§8.1).	Covered

4. Verified research findings (as of 24 Sep 2026)

These facts were checked against primary sources for this document. They constrain the design; several contradict assumptions in the earlier documents.

4.1 ML-KEM parameters (FIPS 203, published 13 Aug 2024)

Parameter set	Category	k	η1 / η2	du / dv	Encaps key	Decaps key	Ciphertext	Secret
ML-KEM-512	1	2	3 / 2	10 / 4	800 B	1,632 B	768 B	32 B
ML-KEM-768	3	3	2 / 2	10 / 4	1,184 B	2,400 B	1,088 B	32 B
ML-KEM-1024	5	4	2 / 2	11 / 5	1,568 B	3,168 B	1,568 B	32 B

- All sets use n = 256, q = 3329. NIST recommends ML-KEM-768 as the default parameter set.
- **Implicit rejection:** Decaps on a modified ciphertext (or with the wrong key) does *not* fail; it returns a pseudorandom secret derived from the ciphertext and a secret value z. Consequence: every "wrong key" and "KEM-ciphertext tamper" test must expect failure at the AES-GCM tag check, not at decapsulation.
- Terminology rule: write **ML-KEM-768**. Use "Kyber" only for history; FIPS 203 ML-KEM is not byte-identical to round-3 Kyber.

4.2 MinIO community edition is archived

Finding — affects assignment Step 1

- MinIO put its open-source repository into maintenance mode in December 2025 and archived it (read-only) on 25 April 2026. GitHub API confirms `archived: true`.
- The last community release is `RELEASE.2025-10-15T17-29-55Z` (a security release). It was published as source only; no official container image accompanied it.
- Reports from September 2026 say the `minio/minio` namespace on Docker Hub no longer serves images; older images reportedly remain on Quay. The spec's `image: minio/minio:<PINNED-TAG>` step may therefore fail.
- Garage (AGPLv3, actively maintained, S3-compatible) is the most-cited like-for-like replacement.

Decision D1 (§5) keeps MinIO, as the assignment requires, by building the final release from source. Because encryption is client-side, the store is swappable, and a Garage fallback costs one adapter configuration change.

4.3 Library status and versions

Component	Verified version / date	Notes
liboqs (C)	0.16.0 — 9 Jul 2026	Exposes "ML-KEM-768". Upstream states it is for prototyping and evaluation, not production.
liboqs-python	0.16.0.1 — 23 Sep 2026	API: KeyEncapsulation, generate_keypair, export_secret_key, encap_secret, decap_secret, get_enabled_kem_mechanisms. Auto-installs liboqs if absent — disable for reproducibility; build liboqs ourselves.
pyca/cryptography	50.0.1	AES-GCM, HKDF, RSA-OAEP, X25519. ML-KEM available since 47/48 with OpenSSL ≥ 3.5 — used as an independent cross-check of liboqs (§11).
MinIO Python SDK	minio 7.2.20	Client library still published on PyPI.
MinIO server	RELEASE.2025-10-15T17-29-55Z	Final community release; source build.
Locust	2.46.6	Optional; not the primary harness (D10).
OpenSSL (dev Mac)	3.6.3	Used for the reference micro-benchmarks below.

4.4 Reference micro-benchmarks (indicative only)

Measured on 24 Sep 2026 with `openssl speed -seconds 2` on the development laptop (Apple M2, OpenSSL 3.6.3, single thread). These are **not results**: they give orders of magnitude for hypothesis setting and for sanity-checking the Python-level timers.

Operation	Time per op	Ops/s	Relevance
ML-KEM-768 keygen	0.035 ms	28,861	Once per recipient; not in timed path
ML-KEM-768 encaps	0.022 ms	45,857	P1 PUT key establishment
ML-KEM-768 decaps	0.035 ms	28,730	P1 GET key establishment
X25519 derive	0.030 ms	33,680	B2 per operation (plus ephemeral keygen on PUT)
RSA-4096 public op (OAEP encrypt)	0.052 ms	~19,400	R1 PUT key establishment
RSA-4096 private op (OAEP decrypt)	3.4 ms	~290	R1 GET key establishment — ~100× ML-KEM decaps
RSA-4096 keygen	270–350 ms	~3	Never inside the timed path
AES-256-GCM, 4 MiB buffers	~0.5 ms per 4 MiB	≈ 7.5 GiB/s	Bulk encryption; far below network cost

What this implies for the hypotheses

- Raw ML-KEM-768 costs tens of microseconds, so the *Python wrapper* (ctypes calls, object allocation, JSON handling) and S3 round-trips will likely dominate measured key-establishment time. The benchmark must report both C-level (liboqs `speed_kem`) and Python-level numbers so readers can separate algorithm cost from binding cost.
- RSA-4096 is expected to be **slower** than ML-KEM-768 on GET because of the private-key operation. That is a legitimate finding and should not be treated as a bug.
- For objects of 1 MiB and larger, network and storage time should dominate; PQC's relative overhead should appear mainly for small objects and in CPU saturation at high concurrency.

4.5 Prior art that bounds our novelty claims

Work	What it does	Consequence for us
Amazon S3 Bucket Keys	A time-limited bucket-level key from KMS derives unique per-object data keys; up to 99% fewer KMS requests.	The closest deployed analogue of N1 (added in v1.4). Bounded reuse with per-object derivation is established practice; see docs/n1-prior-art.md for the resulting claim wording.
AWS Encryption SDK — data key caching (caching CMM)	Reuses a data key across messages, bounded by max_age, max_messages_encrypted and max_bytes_encrypted.	Bounded key reuse with count/age limits is **not new**. C3 must cite it and claim only (a) applying it to per-object PQC KEM roots with per-object HKDF separation, (b) quantifying the performance vs compromise-scope curve, (c) the queue-aware adaptive bound, if the prior-art search confirms.
Prikutse et al., arXiv:2608.13138 (Aug 2026)	AES-256-GCM + hybrid X25519/ML-KEM-768 + ML-DSA-65 for multi-file cloud storage; batch commitment cuts signing from n to 1 (600–700× at 1,000 files).	Closest cloud-storage PQC work; it amortizes signatures. Its abstract reports no S3 store, concurrency or queueing analysis (full-text check pending, Phase 7). Must be cited and contrasted in Related Work.
Rodriguez-Alvarez, arXiv:2508.01694	Performance and storage analysis of ML-KEM vs RSA/ECC in a KEM-DEM scheme (ChaCha20-Poly1305).	Algorithm-level baseline numbers; supports our metadata-size comparisons.
Authenticated PQ session protocol, Frontiers in Physics (2025)	ML-KEM + ML-DSA + AES-256-GCM session protocol with transcript-bound HKDF; LAN/WAN latency.	Session-level key establishment, not object storage; relevant to N1 framing.
TLS-level PQC studies (Cloudflare, OQS profiling)	Hybrid ML-KEM in TLS 1.3 handshakes.	We measure application-layer per-object PQC and hold TLS constant (D6).

Gap we claim: no located work measures per-object ML-KEM-768 envelope encryption *on an S3-compatible object store* under client concurrency up to 500, with matched RSA-4096, X25519 and AES-only baselines, stage-level decomposition and a validated saturation model. D (analysis lead) re-runs this search in Phase 7 before the paper states it.

5. Binding design decisions

Each decision is final unless changed through §1.3. The "Source" column records which input document it confirms or overrides.

ID	Decision	Rationale	Source
D1	Object store = MinIO server built from source at tag RELEASE.2025-10-15T17-29-55Z; record binary SHA-256 and Go version. Dev: multi-stage Dockerfile in repo. Fallback A: a historical image pinned by digest (isolated lab only). Fallback B: Garage, reported as a deviation.	Official images unavailable; the assignment names MinIO; the last release includes a security fix.	Overrides spec §9
D2	Terminology: ML-KEM-768 everywhere; "Kyber" only when citing history.	FIPS 203 naming; ML-KEM differs from round-3 Kyber.	Spec Phase 0

ID	Decision	Rationale	Source
D3	Encapsulation happens on the **client**; the store only ever sees ciphertext. There is no gateway.	Isolates crypto from MinIO internals; matches assignment Step 2.	Overrides draft paper §6.1
D4	RSA-4096 baseline (R1) = RSA-OAEP (SHA-256, MGF1-SHA-256) key transport of a fresh 32-byte random secret per object, then the identical HKDF + AES path. Key pair pre-generated.	"RSA key exchange" in cloud envelope encryption means key wrapping; matched path.	New
D5	X25519 baseline (B2) = ephemeral-static ECDH per object; PUT includes ephemeral keygen; ephemeral public key (32 B) stored in the manifest.	Closest classical analogue of a KEM (DHKEM style).	Refines spec §13
D6	Transport: plain HTTP on an isolated LAN for the main matrix; one sanity cell repeated with TLS. TLS is held constant across modes.	Isolates application-layer PQC; TLS is a confounder.	Draft paper §6.1
D7	All benchmark runs use the **chunked v2 format** for every size (small objects = one chunk). (v1.1: the one-shot v1 format was never built; milestone M1 also uses chunked v2.)	No discontinuity at a size threshold; bounded memory at 500 MiB.	Amends spec §10.3/§11
D8	Replace the full cross-product with three sweeps: concurrency sweep, size sweep and corner points (§9.2).	500 workers × 500 MiB means about 250 GB in flight; infeasible and uninformative.	Amends assignment ranges (confirm with professor)
D9	Storage layout: ciphertext as one S3 object (streamed multipart); manifest written **last** and acts as the commit record. All modes, including B0/B1, write data + manifest.	Two writes instead of three; same atomicity guarantee; matched request count across modes.	Amends spec §10.4
D10	Primary harness = custom multi-process runner (closed-loop and open-loop). Locust optional.	Stage timers must be in-process; the GIL rules out 500 threads in one process.	Amends spec §15
D11	Manifest binary fields encoded as base64url, not hex.	Hex doubles the KEM-ciphertext footprint (2,176 chars vs 1,452) and distorts metadata metrics.	Amends spec §10.3
D12	Pins: Python 3.12.x, liboqs 0.16.0 (own build, shared lib, OQS_DIST_BUILD, OQS_USE_OPENSSL=OFF, OQS_MINIMAL_BUILD = ML-KEM-512/768/1024; flags and commit recorded), liboqs-python 0.16.0.1, cryptography 50.0.1, minio 7.2.20. Ubuntu 24.04 LTS on final VMs.	Reproducibility; no dependency changes during final runs.	Refines spec §8
D13	Developer laptops (macOS/Windows) are for development only. **No reported number** may come from a laptop or from Docker Desktop.	Assignment says Linux instances; VM virtualization noise.	New

6. Reconciliation of the three input documents

6.1 Conflicts between documents and how they are resolved

Topic	Assignment	Spec	Draft paper	Resolution
Classical baseline	RSA-4096	X25519 (B2) only	RSA-4096 or AES-only	Both R1 and B2 (D4, D5)
Concurrency	1–500	1–128	1 to several hundred	1–500 in sweep A (D8)
Object size	1 MB–500 MB	1 KiB–128 MiB	1 MB to hundreds of MB	1 KiB–500 MiB via sweeps
Where KEM runs	Client app	Client	Gateway	Client (D3)
Per-object metadata	—	hex manifest	pk + ct ≈ 2.3 KB	ct 1,088 B + salt + prefix + tags; no pk (§7.5)

Topic	Assignment	Spec	Draft paper	Resolution
Cluster	Cluster on Linux	Single node + optional multi	Multi-node	Single node main; 4-node optional
Harness	—	Locust	—	Custom runner (D10)
Optimizations	—	Bounded session (N1)	Compression, QAT/Nitro offload	N1 only; others are future work

6.2 Errata for the draft paper (must be fixed before reuse)

#	Location	Problem	Fix
E1	§6.1, §8.1	The gateway encapsulates, and the client "generates a Kyber public key and sends it to the gateway". This inverts the KEM roles and contradicts the design.	The recipient owns the key pair; the uploader encapsulates to the recipient's public key on the client (D3).
E2	§7.3	Per-object overhead P + C ≈ 2.3 KB assumes the public key is stored with each object.	Only the 1,088 B ciphertext + 32 B salt + 4 B nonce prefix + 16 B per chunk + manifest are stored; the public key is referenced by a 32 B key id.
E3	Ref. 15	Cites the assignment brief itself.	Remove.
E4	§2.1, throughout	"Kyber-768"; "recently standardized" with no date.	ML-KEM-768; FIPS 203, 13 Aug 2024.
E5	§2.2, §3.2, §3.3	Vendor or blog numbers (+66% / +50 ms handshakes, >700% ClientHello, >99% "encryption strength", 1–10% overhead) stated as fact.	Verify against primary sources or remove. Don't reuse unscientific metrics such as "encryption strength %".
E6	§2.3	"120 ms to 800 ms from 0 to 22 MB" from a low-quality source.	Remove; our own data replaces it.
E7	§3.1	HPC RTT and "4.8–5.0 MB/s effective throughput" figures are unverified.	Check the original paper; otherwise drop.
E8	§5.3, §7.4	Claims compression and hardware offload (QAT, Nitro) as contributions.	We do not implement them. Move to Future Work; claims must match artifacts.
E9	§6.3	Kyber-512/1024 listed as experimental variables.	Out of the core scope; optional extension only.
E10	§6.2	Assumes a MinIO cluster from official images.	Describe the source build and the archival status (D1).

6.3 Amendments to the implementation specification

- **A1** Add mode R1 (RSA-4096 OAEP) to §13 and to all matrices.
- **A2** Replace the §15.2 final matrix with §9.2 of this document (up to 500 workers and 500 MiB).
- **A3** Base64url instead of hex in manifests (D11).
- **A4** The benchmark always uses the chunked v2 format (D7); one-shot v1 was dropped entirely (v1.1).
- **A5** Manifest-as-commit replaces the separate commit marker (D9). B0/B1 also write a manifest.
- **A6** Add a server-side resource sampler (§8.3) and a server-samples schema.
- **A7** Custom multi-process runner (D10); Locust is optional.
- **A8** Wrong-key and KEM-tamper tests expect AES-GCM authentication failure (implicit rejection).
- **A9** Add a cross-implementation test: liboqs encaps ↔ pyca/cryptography (OpenSSL) decaps and vice versa.
- **A10** Replace §9 docker-compose `image:` with a source build (D1).
- **A11** Set liboqs-python so it never auto-downloads liboqs during experiments; fail if the detected liboqs version ≠ 0.16.0.

7. Cryptographic protocol (normative)

7.1 Roles and keys

- **Recipient** owns (ek, dk) ← ML-KEM-768.KeyGen(). ek is distributed; dk never leaves the client host and is stored outside Git with mode 0600.
- **Uploader** knows ek only. **Downloader** holds dk. **Store** (MinIO) holds ciphertext and manifests only.
- One benchmark process may play several roles, but code keeps them in separate objects, so no function receives both ek and dk unless it is a test.
- Baseline keys: RSA-4096 key pair (R1), X25519 static key pair (B2), 32-byte AES master key (B1). All are generated once per experiment by `pqcstore keygen`, outside timed runs.

7.2 Object identity and authenticated context

Identity I = (tenant, bucket, object_key, object_version), where object_version is a fresh UUIDv7 (RFC 9562, time-ordered) per PUT, so the latest committed version is the largest one when listed. The context is canonical JSON (UTF-8, sorted keys, separators "," and ":", no whitespace). Every field that controls derivation or interpretation is in the context and is therefore authenticated.

```
{"aead":"AES-256-GCM","bucket":"pqc-bench","chunk_count":4,"chunk_size":4194304,
 "format":"pqcs-chunked","kdf":"HKDF-SHA256","ke_ct_sha256":"<b64url>","mode":"P1",
 "ke":"ML-KEM-768","object_key":"dataset/item-001","object_version":"<uuid7>",
 "plaintext_size":16777216,"recipient_key_id":"<b64url sha256(ek)>","schema_version":2,
 "tenant":"research"}
```

7.3 PUT — per object (mode P1)

```
(C_kem, S)   <- ML-KEM-768.Encaps(ek)                              # T_KE
salt         <- random(32)
ctx          <- canonical(context incl. sha256(C_kem))
K_obj        <- HKDF-SHA256(IKM=S, salt=salt,
                            info="pqc-s3-object-v2" || 0x00 || ctx)  # T_KDF
prefix       <- random(4)
for j in 0..n-1:                                                   # T_AEAD (summed)
    nonce_j  = prefix || uint64_be(j)                              # 96-bit
    AAD_j    = ctx || uint64_be(j) || uint32_be(len_j) || uint8(is_final_j)
    (c_j,t_j)= AES-256-GCM.Enc(K_obj, nonce_j, P_j, AAD_j)
    stream  += c_j || t_j                                          # streamed to S3 (T_S3)
manifest     = {context: ctx, ke_ct: b64(C_kem), salt: b64(salt),
                nonce_prefix: b64(prefix)}                           # T_SER
PUT data.enc (multipart stream) ; then PUT manifest.json (= commit)
```

7.4 GET — per object (mode P1)

```
GET manifest.json ; parse ; validate schema and identity == requested
check sha256(C_kem) == ke_ct_sha256
S     <- ML-KEM-768.Decaps(dk, C_kem)         # never fails (implicit rejection)
K_obj <- HKDF-SHA256(S, salt, "pqc-s3-object-v2" || 0x00 || ctx)
stream GET data.enc ; for each record of (len_j + 16) bytes:
    verify+decrypt with nonce_j, AAD_j ; reject on tag failure, missing, extra or reordered chunk
    only then release P_j to a temporary sink; publish after the final chunk verifies
```

Streaming rule: a chunk's plaintext may be written to a temporary sink after its own tag verifies, but the caller receives a success result only after the final chunk (is_final = 1) verifies and the count equals chunk_count. On failure the temporary sink is destroyed.

7.5 Stored bytes per object, by mode

Mode	Key-establishment field	Other crypto fields	Approx. crypto metadata (binary)
B0	—	—	0 B (manifest has identity only)
B1	—	salt 32 B, prefix 4 B, 16 B/chunk	36 B + 16 B/chunk
R1	RSA-OAEP ciphertext 512 B	same as B1	548 B + 16 B/chunk
B2	ephemeral X25519 public key 32 B	same as B1	68 B + 16 B/chunk
P1	ML-KEM-768 ciphertext 1,088 B	same as B1	1,124 B + 16 B/chunk
P2	ML-KEM ct 1,088 B + X25519 epk 32 B	same as B1	1,156 B + 16 B/chunk
N1	ML-KEM ct 1,088 B (shared by ≤ B_max objects) + session id 16 B	same as B1	1,140 B + 16 B/chunk (stored per object for self-containment)

Report both the binary crypto metadata above and the actual encoded manifest size (MetadataRatio, §8.1). At 1 KiB objects, P1 metadata exceeds the payload; at 1 MiB it is about 0.1%.

7.6 Mode definitions

Mode	Key establishment per object	Data protection	Purpose
B0	None	None (plaintext, same manifest write)	Raw S3 + harness cost
B1	None — K_obj = HKDF(master key, salt, ctx)	Chunked AES-256-GCM	Symmetric-only baseline (assignment)
R1	RSA-OAEP-4096 wrap/unwrap of random 32 B secret	HKDF + chunked AES-256-GCM	Classical baseline (assignment)
B2	X25519 ephemeral-static ECDH	HKDF + chunked AES-256-GCM	Modern classical baseline
P1	ML-KEM-768 Encaps/Decaps	HKDF + chunked AES-256-GCM	Main PQC mode
P2	X25519 + ML-KEM-768; IKM = S_kem || S_ecdh; both ct hashes in ctx	HKDF + chunked AES-256-GCM	Hybrid migration mode (research combiner; cite X-Wing draft as the standardized direction)
N1	ML-KEM-768 once per session root, ≤ B_max objects and ≤ T_max s	Per-object HKDF + chunked AES-256-GCM	Candidate optimization (C3)

7.7 Nonce, key and secret-handling rules

- Every object gets a fresh K_obj (per-object KEM secret or per-object salt/context), so nonce_j = prefix || counter is unique under each key; counter overflow is impossible at 4 MiB chunks. (NIST SP 800-38D requires IV uniqueness per key.)
- N1: K_i is distinct per object because salt_i and I_i are distinct; nonce rules apply per K_i.
- Never log or persist S, K_obj, dk, RSA private keys or plaintext. The logger has an allow-list of fields; tests assert that secrets never appear in log output.
- Python cannot guarantee memory erasure. Use bytearray where the API allows, overwrite after use, and document the limitation.

8. Metrics, timers and records

8.1 The four assignment metrics — exact definitions

Assignment metric	Operational definition	Raw fields	Reported as
Cryptographic handshake overhead (ms)	$T_{KE} + T_{KDF}$ for one object. PUT: encaps / RSA wrap / ECDH incl. ephemeral keygen. GET: decaps / RSA unwrap / ECDH. B1: T_{KDF} only; B0: 0. Also Δ vs B1.	ke_ns, kdf_ns	Median, p95, p99 per mode × op × size × concurrency, bootstrap CI over runs
Data throughput (MB/s)	Σ plaintext bytes of successful ops in the measured interval ÷ interval length (aggregate, all workers). Reported in MiB/s; ops/s alongside.	object_size_bytes, success, timestamps	Median across runs + 95% CI
Client / server CPU utilization (%)	Client: Σ CPU of runner process tree ÷ (interval × logical cores) × 100. Server: MinIO process likewise. Also CPU-ms per successful request (thread CPU time).	client/server samples; cpu_thread_ns	Mean and p95 of 1 Hz samples; CPU-ms/request
Memory footprint growth (MB)	Peak RSS during the interval − RSS at the end of warm-up, for the client process tree and for MinIO. Plus crypto metadata bytes per object.	samples; metadata_bytes	MiB per cell; bytes per object

8.2 Timer decomposition

```
T_PUT = T_KE + T_KDF + T_AEAD_ENC + T_SER + T_S3_PUT(data) + T_S3_PUT(manifest) + T_other
T_GET = T_S3_GET(manifest) + T_PARSE + T_KE + T_KDF + T_S3_GET(data) + T_AEAD_DEC + T_other
```

- time.perf_counter_ns() for wall stages; time.thread_time_ns() for per-request CPU.
- Because chunks stream, T_AEAD and T_S3 interleave. T_AEAD = Σ chunk crypto time; T_S3 = total wall − all other measured stages. T_other must be < 2% of total in validation (Phase 5 exit test).
- Payloads come from a pre-generated 64 MiB random pool (memoryview slices, cycled for larger objects), so no disk reads and no payload generation happen inside timers.
- No per-request printing during runs. Records are buffered and flushed per worker at run end.
- Every process calls envelope.warm_up() before its first timed operation: the first liboqs call pays ~70 ms of library loading and the first HKDF call ~3 ms of setup, which would otherwise be booked as key-establishment and KDF time.

8.3 Record schemas

requests.jsonl — one line per request

```
experiment_id, run_id, block_id, request_id, worker_id, pid, ts_start_utc_ns,
git_commit, config_hash, machine_id, mode, operation, object_size_bytes,
concurrency, loop_type(closed|open), target_rate, queue_wait_ns,
ke_ns, kdf_ns, aead_ns, ser_ns, s3_ns, total_ns, cpu_thread_ns,
chunk_size, chunk_count, bytes_sent, bytes_received, metadata_bytes,
session_id, session_bmax, session_count,            # N1 only
success, error_class
```

client-samples.csv / server-samples.csv — 1 Hz

```
run_id, ts_utc_ns, host, proc_cpu_pct, host_cpu_pct, proc_rss_bytes,
host_mem_used_bytes, disk_read_bytes, disk_write_bytes, net_rx_bytes, net_tx_bytes, load_avg_1m
```

run-manifest.yaml — one per run

CPU model, physical/logical cores, RAM, kernel, Python, compiler, liboqs version + commit + CMake flags, liboqs-python version, cryptography version and OpenSSL backend, MinIO binary SHA-256 and Go version, iperf3 bandwidth, fio disk baseline, NIC, MTU, TLS on/off, configuration hash, randomized block order, start/end times, and validity flags.

8.4 Derived metrics

```
Overhead_X/Y (%) = 100 * (T_X - T_Y) / T_Y          # e.g. P1 vs B2, P1 vs R1, P1 vs B1
KEMShare         = T_KE / T_total
MetadataRatio    = (manifest_bytes + tag_bytes) / plaintext_bytes
CPU_per_req      = cpu_thread_ns / successful requests
Error rate       = failed / attempted   (failures are never dropped from raw data)
```

9. Experiment design

9.1 Pre-registered research questions and hypotheses

Hypotheses are recorded here **before** final runs and reported as supported or not supported. Thresholds are working values; the pilot may refine them, but only before Phase 8.

ID	Question / hypothesis	Test
RQ1	How much does ML-KEM-768 add to median/p95/p99 PUT and GET latency vs B1, R1 and B2?	Sweeps A, B
H1	For objects ≥ 1 MiB at concurrency 1, |Overhead P1/B2| < 5% on median latency.	Sweep B
H2	KEMShare decreases monotonically with object size and exceeds 10% only at ≤ 64 KiB.	Sweep B
H3	R1 GET latency > P1 GET latency at all sizes ≤ 1 MiB (RSA private-key op).	Sweeps A, B
RQ2	How does relative overhead change with size, operation mix and concurrency?	Sweeps A, C
H4	At high concurrency, the P1 vs B1 CPU-ms/request gap stays roughly constant in absolute terms and shrinks relative to total latency.	Sweep A
RQ3	At what offered load does the crypto client saturate, and how accurately does a queueing model predict it?	Sweep D
H5	Predicted saturation throughput is within ±20% of observed for ≤ 1 MiB objects.	Sweep D
RQ4	Can bounded KEM reuse (N1) reduce overhead within an explicit compromise-scope budget?	Sweep E
H6	N1 with B_max = 100 recovers ≥ 80% of the P1 − B1 latency gap for ≤ 64 KiB objects.	Sweep E

9.2 Experiment matrix (replaces the spec's final matrix)

Sweep	Sizes	Concurrency	Ops	Modes	Loop / unit	Cells
A — concurrency	64 KiB, 1 MiB, 16 MiB	1, 8, 32, 64, 128, 256, 500	PUT, GET	B0 B1 R1 B2 P1	closed; fixed duration	3×7×2×5 = 210
B — size	1 KiB, 64 KiB, 1 MiB, 16 MiB, 128 MiB, 500 MiB	1, 8	PUT, GET	B0 B1 R1 B2 P1	closed; fixed op count	6×2×2×5 = 120
C — corners + mix	128 MiB @ 32, 64; 500 MiB @ 16, 32; 1 MiB & 16 MiB @ 1–128 with 70/30 GET/PUT	as listed	PUT, GET, mix	B1 R1 P1	closed	≈ 45
D — open loop	1 KiB, 64 KiB, 1 MiB	λ = 0.3…1.1 × measured capacity (7 levels)	PUT	B1 P1	open; Poisson	3×7×2 = 42
E — N1	1 KiB, 64 KiB, 1 MiB	1, 32, 128	PUT, GET	N1 with B_max in {1, 10, 100, 1000}; T_max = 60 s	closed	3×3×2×4 = 72
P2 (hybrid)	Sweep A at 1 MiB only	all of Sweep A	PUT, GET	P2	closed	14

9.3 Run protocol

- **Unit of replication = one independently restarted run** (fresh runner processes; MinIO kept up within a block, restarted between blocks). Default 5 runs per cell; raise to 10 for cells where the pilot shows a run-level coefficient of variation above 5%.
- Closed-loop fixed-duration cells (Sweep A): 10 s warm-up discarded + 30 s measured. Fixed-count cells (Sweep B, large objects): 2 warm-up ops + max(20 ops, 30 s). The two definitions are never mixed within one figure.
- Blocks = (size, concurrency, operation). Mode order is randomized within each block with a recorded seed.
- GET cells read from a pre-populated set per (mode, size) written by an untimed setup phase; PUT cells delete their objects after the run. Page-cache state is documented (warm by default; optional drop-caches variant).
- Validity flags (declared now): runner host CPU > 90% → flag *client-bound* (kept; it is the phenomenon of RQ3); infrastructure errors (connection refused, disk full, VM reboot) → run invalid and rerun, with the reason logged. No other outlier removal.

9.4 Time and resource budget

Item	Estimate	Consequence
Sweep A	210 cells × 5 runs × ~50 s ≈ 14.6 h	Two overnight batches
Sweeps B + C	≈ 10 h (large objects bound by network: 500 MiB ≈ 4.4 s at 1 Gb/s, 0.45 s at 10 Gb/s)	Prefer 10 GbE; record iperf3
Sweeps D + E + P2	≈ 6 h	One batch
Client memory at 500 workers	Measured in Phase 4 (dev laptop): ~40–52 MiB peak RSS growth per in-flight PUT+GET for objects ≥ 16 MiB, flat up to 500 MiB (8 MiB parts, 4 MiB chunks); ~3 MiB at 1 MiB objects	Up to ~25 GB at 500 concurrent large transfers: Sweep A at 16 MiB and Sweep C corners need a 32–64 GB client VM; re-measure on the Linux VM (glibc returns large blocks sooner than macOS)
Server disk	GET sets: 20 objects × 5 modes × 500 MiB ≈ 50 GB (+ smaller sizes)	≥ 200 GB dedicated SSD

10. Infrastructure and measurement environment

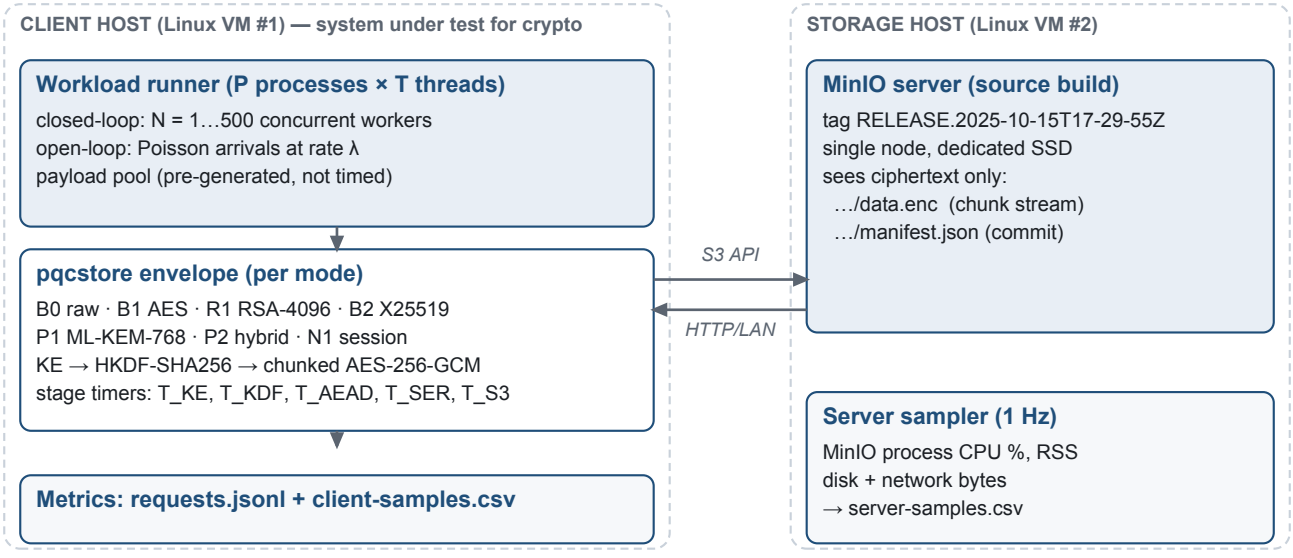


Figure 1 — Two-host topology for final measurements.

Host	Minimum	Preferred	Notes
Client VM	8 vCPU, 32 GB RAM	16 vCPU, 64 GB RAM	Runs crypto + runner; CPU is the resource under study
Storage VM	4 vCPU, 16 GB RAM, 200 GB SSD	8 vCPU, 32 GB, NVMe	MinIO only; dedicated data disk
Network	1 GbE, same subnet	10 GbE, same placement group	Record iperf3 before every batch
OS	Ubuntu 24.04 LTS	same	Performance CPU governor where available; NTP synced; no other workloads

- **Fallback if only one Linux machine is available:** pin MinIO and the runner to disjoint CPU sets (taskset/cgroups) and report this as a threat to validity.
- **Development:** any laptop with Docker; MinIO from the repo's multi-stage Dockerfile. Development numbers are never reported (D13).
- Concurrency model: P = min(N, client logical cores) worker processes; exactly N worker threads are distributed across them (difference between processes at most 1), so total concurrency is exactly N. Crypto calls release the GIL only inside C code; multiple processes give real parallelism.
- Server sampler: a small psutil agent on the storage VM, started and stopped by the runner over SSH, writing server-samples.csv. MinIO's Prometheus endpoint is optional corroboration.

11. Correctness and security tests (gate for all measurements)

Performance data from a build with any failing test is invalid. These tests run in CI and via `make test` before every batch.

Test	Procedure	Expected result
KEM agreement	Encaps with ek, decaps with dk	Equal 32 B secrets
KEM cross-implementation	liboqs encaps → pyca/cryptography decaps and reverse	Equal secrets
Wrong private key	Decrypt with a second key pair	AES-GCM tag failure (not a decaps error)
Round trip (all modes)	Encrypt, upload, download, decrypt	SHA-256 equal
Empty object	0-byte round trip	Success; one final chunk
Boundary sizes	1 B, chunk−1, chunk, chunk+1, 2×chunk	Success, no truncation
Ciphertext tamper	Flip one bit in any chunk	Rejection; no plaintext released
Manifest tamper	Change bucket, key, version, size, mode or chunk_size	Rejection
KEM ciphertext tamper	Flip one bit of C_kem	Hash-binding or tag failure
Manifest swap	Exchange two objects' manifests	Rejection
Chunk deletion / reorder / append	Edit the stored stream	Rejection
Truncation	Drop the final chunk	Rejection (is_final flag + count)
Replay	Present an old version as the current version	Rejection via identity binding
Partial upload	Kill between the data and manifest writes	Reader ignores the version; GC removes the orphan
Concurrent writers	Same logical key, distinct versions, 64 writers	No collisions; all versions decrypt
Nonce uniqueness	10^6 chunk nonces under one key	No duplicates
N1 bounds	Stress at 500 workers with B_max and T_max	Never more than B_max objects per root; never older than T_max
Secret hygiene	Grep logs and records for key material	None found

Test	Procedure	Expected result
Streaming memory	Round trip of 16, 128, 500 MiB	Peak RSS growth independent of size (± buffers)
Timer arithmetic	Σ stages vs total	T_other < 2% of total

12. Analysis plan

12.1 Statistics

- Per run: median, p95, p99 latency; throughput; CPU and memory summaries. Across runs: the median of run-level statistics, with a 95% bootstrap CI (10,000 resamples over runs, never over individual requests).
- Overheads (e.g. P1 vs B2) are computed as paired ratios within the same block, with a bootstrap CI. Report absolute differences (ms), ratios and CIs; p-values are optional and never shown alone.
- Report per cell: the number of runs, measured duration, successful ops, failed ops by error_class, and validity flags.
- `analysis/validate.py` rejects files with schema errors, checksum mismatches or timer-arithmetic violations before aggregation.

12.2 Queueing model (RQ3)

```
Crypto station:  m = client worker processes (≈ cores);  E[S_cpu] = mean CPU time per request
Utilization:     rho = lambda * E[S_cpu] / m
Capacity:        X_max = min( m / E[S_cpu],  B_net / bytes_per_req,  server capacity from B0 )
Waiting (M/M/m): Erlang-C formula
Waiting (M/G/m): Allen-Cunneen  Wq ~ Wq(M/M/m) * (1 + CV_s^2) / 2
```

Fit E[S] and CV² from Sweep A low-concurrency cells (not from the cells being predicted). Compare predicted and observed saturation throughput and latency-vs-λ curves in Sweep D, and report the prediction error. Do not present an M/M/m fit as evidence that arrivals or service times are exponential.

12.3 Planned figures and tables

ID	Content	Answers
F1	Latency CDF by mode (1 MiB, concurrency 1, PUT and GET)	RQ1
F2	Median and p99 latency vs concurrency, one panel per size, lines = modes	RQ1, RQ2
F3	Overhead % (P1 vs B1, B2, R1) vs object size	H1, H3
F4	Stacked stage breakdown (KE, KDF, AEAD, SER, S3) vs size	H2
F5	Throughput vs concurrency with the predicted X_max line	RQ3, H5
F6	Client and server CPU % vs concurrency; CPU-ms per request	H4, R10
F7	Memory growth vs concurrency and size	R11
F8	N1: latency saving vs compromise scope (B_max) — Pareto plot	RQ4, H6
T1	Crypto metadata bytes per object by mode, with cloud-scale projection	Storage overhead
T2	C-level (liboqs speed_kem / openssl speed) vs Python-level KE time	Binding overhead
T3	Environment and version manifest	Reproducibility

13. Repository and tooling

```
pqc-object-store/
  README.md  pyproject.toml  requirements-lock.txt  Makefile  .env.example  .gitignore
  docker/minio.Dockerfile          # multi-stage source build of RELEASE.2025-10-15T17-29-55Z
  docker-compose.yml               # dev only
  config/  development.yaml  pilot.yaml  final.yaml
  src/pqcstore/
    cli.py  settings.py
    crypto/  modes.py (B0 B1 R1 B2 P1 P2 N1)  mlkem.py  rsa_oaep.py  x25519.py
             kdf.py  chunked_aead.py  manifest.py  session.py (N1)
    storage/ minio_store.py        # data + manifest-as-commit, GC of orphans
    benchmark/ payload.py  runner.py (closed/open loop)  metrics.py
               client_sampler.py  server_agent.py  orchestrate.py
  tests/  test_kem.py  test_cross_impl.py  test_roundtrip.py  test_tamper.py
          test_chunking.py  test_manifest_binding.py  test_session_bounds.py
          test_minio_integration.py  test_secret_hygiene.py
  analysis/  validate.py  aggregate.py  statistics.py  queueing.py  figures.py
  experiments/<experiment_id>/  run-manifest.yaml  requests.jsonl.zst
          client-samples.csv.zst  server-samples.csv.zst  checksums.sha256  logs/
  results/  processed/  figures/
  paper/     # LaTeX source; tables and figures generated from results/
```

Make targets: `setup`, `minio-build`, `services-up`, `test`, `smoke`, `pilot`, `final-experiment CONFIG=…`, `validate-results`, `analyze`, `figures`, `paper`. Keys, credentials and raw experiment data are git-ignored; a small non-sensitive raw sample plus all processed tables are committed for the artifact.

14. Team roles and schedule

14.1 Ownership (4 students)

Role	Owns (code)	Owns (paper)	Reviews
A — Crypto lead	crypto/*, all modes, keygen CLI, negative and cross-implementation tests, secret hygiene	Threat model, protocol, security discussion	C's code
B — Storage & streaming lead	MinIO source build, Dockerfile, VM provisioning, minio_store, chunked streaming, GC, integration tests, iperf3/fio baselines	System design, environment, threats to validity	D's code
C — Benchmark lead	runner (closed/open loop), payload pool, metrics records, client and server samplers, orchestration, configs, checksums	Methodology, experiment design	A's code
D — Analysis & paper lead	validate/aggregate/statistics, queueing model, figures, prior-art search	Introduction, related work, results, model, integration	B's code

14.2 Schedule (relative weeks — align W10 with the submission deadline)

Week	Phase	Deliverable / exit criterion	Lead
W1	0 Freeze protocol · 1 Environment	SoT approved; MinIO built from source; liboqs 0.16.0 example runs; raw 1 MiB S3 round trip SHA-256 match	B, A
W2	2 Crypto MVP	**Milestone M1**: keygen → put 1 MiB → get → sha256 match; tamper rejected; stage timings printed; all local crypto tests pass	A
W3	3 S3 integration + baselines	Manifest-as-commit; B0 B1 R1 B2 P1 round-trip; 100 consecutive end-to-end runs without orphans	A, B
W4	4 Streaming	Chunked v2; 500 MiB round trip with size-independent memory	B
W5	5 Instrumentation	Runner, samplers, schema-valid records; timer arithmetic check; Linux VMs ready	C
W6	6 Pilot	Pilot matrix run; saturation understood; repetitions and durations fixed; hypothesis thresholds final	C, D

Week	Phase	Deliverable / exit criterion	Lead
W7	7 Candidate algorithm	N1 fixed-bound mode + P2; bounds never exceeded under stress; prior-art search written up	A, D
W8	8 Final measurements	Code frozen and tagged; randomized batches; raw files checksummed and backed up	C
W9	Analysis	All figures and tables regenerated from raw data; queueing validation	D
W10	Paper + artifact	Paper complete; a second team member reproduces the qualitative findings from a clean checkout	All

Paper writing runs in parallel from W1 (related work, method, threat model), so W9–W10 only add results and discussion.

15. Definition of done

- All unit and integration tests pass from a clean checkout on the final VMs.
- ML-KEM-768 is confirmed enabled at runtime; liboqs version = 0.16.0; no auto-download occurred.
- No private material in Git history or logs (automated check).
- The chunked v2 format is versioned, and every interpretation field is authenticated.
- All seven modes use the same data path and timer definitions.
- The client and server samplers ran for every run; load-generator saturation is flagged.
- Every final cell has ≥ 5 independent runs; raw files are immutable and checksummed.
- Every number, table and figure in the paper is regenerated by a script from raw files.
- Prior-art search completed; novelty claims worded to match it (§4.5).
- Limitations state that liboqs and this prototype are not production cryptographic services, and that MinIO CE is archived.

16. Risks, threats to validity and open questions

16.1 Risk register

Risk	Likelihood	Mitigation
MinIO source build fails or needs a newer Go	Medium	Pinned Go toolchain in the Dockerfile; fallback image by digest; Garage fallback
No access to two Linux VMs	Medium	Ask the department/cloud credits in W1; single-host CPU-pinned fallback
500 workers exhaust client memory	Medium	Streaming, 8 MiB parts, 32–64 GB client VM, corner points limited by D8
Network masks crypto effects at large sizes	High (expected)	Report as a finding; small-object and CPU metrics carry the PQC signal
Python overhead dominates KEM cost	High (expected)	Report C-level vs Python-level (T2); state this clearly
N1 novelty overlaps existing caching	High	Cite the AWS SDK; claim only the PQC-specific evaluation and the queue-aware policy
Schedule slip	Medium	Core first (B0, B1, R1, P1, Sweeps A/B); P2, N1 and Sweep D are droppable in that order

16.2 Threats to validity (to be written into the paper)

- Single hardware platform and a single implementation (liboqs via Python); results may not transfer to optimized native clients or hardware offload.
- Single-node MinIO without erasure coding; distributed deployments add server-side costs we do not measure.

- Warm page cache on GETs; plain HTTP transport; LAN latency far below WAN.
- Virtualized hosts: noisy neighbours are mitigated with repeated runs and randomization, not eliminated.
- MinIO CE is archived; the conclusions concern S3-compatible stores generally, and the store is swappable.

16.3 Questions to confirm with the professor (W1)

#	Question	Our proposed answer
Q1	May we replace the full 1–500 × 1–500 MB cross-product with the three sweeps?	Yes, per D8, justified by the resource budget
Q2	Is MinIO built from its final source release acceptable, given the archival? Is Garage acceptable if the build fails?	Source build (D1)
Q3	Does "RSA-4096 key exchange" mean RSA-OAEP key transport per object?	Yes (D4)
Q4	Is "handshake overhead" per-object key establishment rather than a TLS handshake?	Yes (§8.1, D6)
Q5	Paper format, page limit and deadline; is an artifact expected?	Assume IEEE 2-column, 8–10 pages + artifact
Q6	Are department VMs or cloud credits available?	Request 2 VMs

17. References

[1] NIST, FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard, 13 Aug 2024. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf

[2] NIST, SP 800-38D: Recommendation for Block Cipher Modes of Operation: GCM and GMAC, 2007. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf

[3] H. Krawczyk, P. Eronen, RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function (HKDF), 2010.

[4] K. Moriarty et al., RFC 8017: PKCS #1 v2.2 (RSA-OAEP), 2016.

[5] A. Langley, M. Hamburg, S. Turner, RFC 7748: Elliptic Curves for Security (X25519), 2016.

[6] Open Quantum Safe, liboqs v0.16.0 (9 Jul 2026). https://github.com/open-quantum-safe/liboqs

[7] Open Quantum Safe, liboqs-python v0.16.0.1 (23 Sep 2026). https://github.com/open-quantum-safe/liboqs-python

[8] pyca/cryptography changelog (ML-KEM since 47.0.0/48.0.0; current 50.0.1). https://cryptography.io/en/stable/changelog/

[9] Trail of Bits, "Shipping post-quantum cryptography to Python", 30 Jun 2026. https://blog.trailofbits.com/2026/06/30/shipping-post-quantum-cryptography-to-python/

[10] MinIO repository (archived) and release RELEASE.2025-10-15T17-29-55Z. https://github.com/minio/minio

[11] MinIO issue #21714, "Maintenance Mode". https://github.com/minio/minio/issues/21714

[12] Storm Developments, "MinIO's community edition is archived. What still runs in 2026". https://stormdevelopments.ca/blog/minio-s-community-edition-is-archived-what-still-runs-in-2026/

[13] MinIO Python SDK 7.2.20. https://pypi.org/project/minio/

[14] Garage S3-compatible object store. https://garagehq.deuxfleurs.fr/

[15] AWS Encryption SDK Developer Guide, "Data key caching details". https://docs.aws.amazon.com/encryption-sdk/latest/developer-guide/data-caching-details.html

[16] L. F. Prikutse et al., "A Commitment-Based Hybrid Post-Quantum Cryptographic Model for Multi-File Cloud Storage", arXiv:2608.13138, Aug 2026.

[17] N. Rodriguez-Alvarez, "Performance and Storage Analysis of CRYSTALS-Kyber (ML-KEM) as a Post-Quantum Replacement for RSA and ECC", arXiv:2508.01694.

[18] "Design and implementation of an authenticated post-quantum session protocol using ML-KEM, ML-DSA and AES-256-GCM", Frontiers in Physics, 2025. doi:10.3389/fphy.2025.1723966

[19] D. Connolly, P. Schwabe, B. Westerbaan, "X-Wing: general-purpose hybrid post-quantum KEM", IETF CFRG Internet-Draft draft-connolly-cfrg-xwing-kem.

[20] M. Harchol-Balter, Performance Modeling and Design of Computer Systems, Cambridge University Press, 2013 (M/M/k, Erlang-C, M/G/k approximations).

[21] ACM, Artifact Review and Badging, Version 1.1. https://www.acm.org/publications/policies/artifact-review-and-badging-current

[22] Locust 2.46.6 documentation, distributed load generation. https://docs.locust.io/

[23] Team 12 inputs: Assignment brief (Team12_Paper_Assignment); Implementation Specification v1.0; draft paper "Benchmarking PQC in Cloud Object Stores under Scaled Client Concurrency".